M4010104 / M4020104: PROGRAMMING IN PYTHON · DIGITAL UNIVERSITY KERALA · CAPSTONE PROJECT SPECIFICATION

Project 09: Malayalam Multi-Domain Text Summarization Compression & Extractive Scorer

Effort Budget: 10–12 Hours | Core Stack: Pure Python, NumPy, Pandas, Matplotlib, Seaborn, Plotly | Max Marks: 25 | Instructor: Dr. Sunil TT

1. Executive Overview & Domain Context

Automated text summarization in Malayalam spans diverse literary and journalistic registers—including contemporary news reporting, social media micro-posts, and classical scriptural passages. Benchmarking summarization algorithms requires evaluating compression ratios, lexical retention, novel word introduction, and extractive sentence coverage.

Your team will construct a multi-domain summarization benchmarking suite in pure Python that parses paired source-summary documents, audits text length consistency, implements a sentence-level TF-IDF extractive ranking engine in NumPy, and computes unigram/bigram token overlap metrics answering the 12 core questions below.

Primary Public Dataset & Telemetry/Data Schema

The project utilizes the verified public Malayalam Texts-Summary Dataset on Kaggle (**`https://www.kaggle.com/datasets/nigeljoetensing/malayalam-texts-summary-dataset`**). The dataset structure captures:

- **Raw Feature Attributes:** **`domain`** (News, Tweets, Bible, Quran), **`source_text`**, **`summary_text`**.
- **Engineered / Target Variables:** **`source_words`**, **`summary_words`**, **`compression_ratio`** (summary_len / source_len), **`extractive_score`**, **`token_f1_overlap`**.

2. Complete Project Directory & File Architecture

Each team must organize their repository into a standardized, production-grade layout. All application source code must reside inside a dedicated package (**`src/`**), with separate directories for raw/curated data, interactive development notebooks, visual figures, and formal documentation:

```
malayalam-summarization-evaluator/
├── README.md                          # Repository documentation & GitHub Pages source
├── requirements.txt                   # Dependency specification (numpy, pandas, matplotlib, seaborn, p
├── run_pipeline.py                    # Standalone production CLI application entry point
├── data/
│   ├── raw/
│   │   └── malayalam_summaries_raw.csv            # Raw uncleaned input dataset (including simulated/in
│   └── processed/
│       ├── clean_malayalam_summaries.csv          # Curated, validated clean CSV dataset
│       └── summarization_audit.log           # Timestamped audit report detailing filtered/rejected rec
├── src/
│   ├── __init__.py                    # Python package initialization
│   ├── ingestion.py                   # Defensive line-by-line file reader, range auditor & CSV exporte
│   ├── analyzer.py                    # MalayalamSummaryEvaluator OOP class, NumPy vectorization & Pand
│   └── visualizer.py                  # Matplotlib envelope plots, Seaborn heatmaps & Plotly HTML expor
├── notebooks/
│   └── development_cycle.ipynb        # Development cycle notebook: EDA, trials, tests & unit assertion
└── reports/
    ├── figures/
    │   ├── primary_envelope.png       # Matplotlib confidence envelope & regression overlay
    │   ├── correlation_heatmap.png    # Seaborn annotated feature correlation heatmap
    │   └── distribution_regimes.png   # Seaborn distribution across categorical/domain tiers
```

```
    ├── interactive_dashboard.html     # Standalone interactive Plotly HTML dashboard with controls
    └── Project_Report.pdf             # 3-4 page final technical report
```

3. Dual Development Deliverables: Notebook vs. Standalone Program

To mirror industry software engineering standards, each student group must deliver two distinct, complementary implementations of their solution:

Deliverable A: The Development Cycle Notebook (`notebooks/development_cycle.ipynb`)

The development notebook chronicles your team's engineering journey from raw data exploration to validated model prototypes. It is not merely a final script pasted into cells, but an authentic record of the development cycle:

- **Phase 1: Exploratory Data Ingestion & Diagnostics:** Loading raw data, inspecting missing values, testing boundary rules, and logging rejected records.
- **Phase 2: Mathematical Formulation & Feature Verification:** Developing and validating NumPy array calculations against target benchmarks using `np.allclose()`.
- **Phase 3: OOP Class Architecture & Unit Tests:** Instantiating the `MalayalamSummaryEvaluator` class, executing automated unit assertions (e.g., verifying that passing invalid parameters raises an explicit `ValueError`), and testing edge cases.
- **Phase 4: Statistical Modeling & Regimen Exploration:** Implementing ordinary least-squares regression, rolling-window calculations, and multi-metric Pandas grouping.
- **Phase 5: Visual Prototype Development:** Iteratively designing and refining Matplotlib and Seaborn figures, followed by configuring the interactive Plotly HTML dashboard.

Deliverable B: The Final Standalone Python Program (`run_pipeline.py` & `src/`)

While the notebook documents the exploratory development cycle, your repository must also provide a production-ready, fully executable Python application. The program must run end-to-end directly from the terminal without requiring Jupyter:

- **Command-Line Interface (CLI):** Executed via `python run_pipeline.py --input data/raw/malayalam_summaries_raw.csv --output-dir data/processed --report-dir reports`.
- **Modular Package Import:** The driver script imports cleanly separated domain logic from `src.ingestion`, `src.analyzer`, and `src.visualizer`.
- **Terminal Feedback & Exit Codes:** The program must print structured progress indicators, log clean/faulty record counts to the console, and return an exit code of `0` upon successful pipeline completion.
- **Automated Artifact Generation:** In a single execution pass, the script reads raw inputs, writes `data/processed/clean_malayalam_summaries.csv`, saves `data/processed/summarization_audit.log`, exports static figures to `reports/figures/`, and generates the standalone `reports/interactive_dashboard.html`.

4. Technical Investigation Requirements & Research Questions (Q1–Q12)

Your team's implementation across the development notebook and standalone program must rigorously answer the following 12 course-aligned engineering questions:

Course Topic & Script	Q#	Concrete Engineering & Analytical Task	Python Libraries & Methods
File I/O & Exceptions (`src/ingestion.py`)	Q1	**Document-Summary Pair Validation Audit:** Parse multi-domain summary CSV lines with try-except. Validate that both source and summary fields are non-empty, summary length is strictly less than source length, and log faulty pairs.	open(), try-except (ValueError, IndexError), text length comparison

Course Topic & Script	Q#	Concrete Engineering & Analytical Task	Python Libraries & Methods
Data Curation & Export (`src/ingestion.py`)	**Q2**	**Curated Summarization Benchmark Export & Audit:** Export validated document pairs to clean_malayalam_summaries.csv. Concurrently log empty summaries, identical source-summary pairs, and corrupted lines in summarization_audit.log.	Defensive file writing, dictionary parsing, formatted log reporting
OOP Design (`src/ingestion.py`)	**Q3**	**Malayalam Summary Evaluator Class:** Design an encapsulated class MalayalamSummaryEvaluator with methods load_from_csv(), score_extractive_sentences(top_k=2), and summary_report(). Ensure top_k <= 0 raises ValueError.	OOP class, __init__, encapsulation, raise ValueError
NumPy Arrays (`src/analyzer.py`)	**Q4**	**Vectorized Compression Ratios & Verification:** Using only raw NumPy arrays of word counts, recompute compression ratios (Summary_Len / Source_Len) and novel word percentages. Verify scores via np.allclose().	np.divide(), array broadcasting, np.allclose()
NumPy Masking (`src/analyzer.py`)	**Q5**	**High-Compression Outlier Masking & Overlap:** Flag document pairs with extreme compression ratios (< 0.15 vs. > 0.60) in NumPy. Identify whether high-compression documents exhibit significantly lower lexical retention via set overlap.	np.mean(), np.std(), boolean masks (&, |), Python set
NumPy Regression (`src/analyzer.py`)	**Q6**	**Source Length vs. Summary Length OLS Fit:** Fit an ordinary least-squares model (Summary_Len = b0 + b1*Source_Len) across domains via np.linalg.lstsq. Quantify domain-specific compression slopes and residual errors.	np.column_stack(), np.linalg.lstsq(), matrix product (@), R2
Pandas Series (`src/analyzer.py`)	**Q7**	**Domain Compression Distribution Audit:** Quantify document counts and average compression ratios across the 4 domains (News, Tweets, Bible, Quran) in Pandas. Audit standard deviation of summary lengths.	.groupby('domain').agg(), .describe()
Pandas Groupby (`src/analyzer.py`)	**Q8**	**Domain-by-Length Discretization & Pivot Table:** Discretize source length into Short, Medium, and Long. Build a 4×3 pivot_table(margins=True) displaying mean compression ratio across domain and length tiers.	pd.cut(), .groupby().agg(), .pivot_table(margins=True)
Pandas Reshaping (`src/analyzer.py`)	**Q9**	**Wide-to-Long Token Overlap Ranking via Melt:** Reshape evaluation metrics (Precision, Recall, F1-Score) into tidy long format via pd.melt() and extract the top 5 highest scoring summaries per domain.	pd.melt(), .groupby('domain'), .nlargest(5)
Matplotlib (`src/visualizer.py`)	**Q10**	**Matplotlib Source vs. Summary Length Scatter:** Plot Source Length vs. Summary Length across domains with Matplotlib. Overlay the OLS regression trendline and compression envelope lines with comprehensive legends.	plt.subplots(), ax.scatter(), ax.plot(), custom legends
Seaborn (`src/visualizer.py`)	**Q11**	**Seaborn Domain Compression Violin & KDE Plots:** Render side-by-side Seaborn violin plots comparing compression ratio distributions across the 4 domains. Overlay KDE density curves with sns.kdeplot().	sns.violinplot(), sns.kdeplot(), sns.despine()
Plotly HTML (`src/visualizer.py`)	**Q12**	**Interactive Summarization Evaluation Dashboard:** Build an interactive HTML dashboard (interactive_dashboard.html) displaying document pairs, compression ratios, and F1 scores with domain dropdown filters.	plotly.express.scatter(), custom hover tooltips, write_html()

5. Collaborative Git Workflow, GitHub Pages & README.md Specification

Best Practices: Collaborative Git Workflow & GitHub Pages Deployment from Day 1

From the very first day, the student team must adopt professional version control and documentation practices. On Day 1, one designated team member initializes the central public GitHub repository, establishes a clear branch protection rule preventing direct pushes to **`main`**, adds all teammates as registered **Contributors** (with write access), and grants read access to the Course Instructor (**`Dr. Sunil TT`**) and teaching assistants. Immediately thereafter, the team must activate **GitHub Pages** (via *Settings → Pages → Build and deployment: Deploy from a branch: main /docs or root*) so that project documentation, interactive architecture diagrams, and the live Plotly dashboard (**`reports/interactive_dashboard.html`**) are publicly accessible and continuously deployed throughout the development lifecycle. All feature development must follow a strict branching model (e.g., **`feature/ingestion`**, **`feature/analysis`**, **`feature/visualization`**): individual team members must write small, atomic, well-commented commits from their own personal GitHub accounts and merge them into **`main`** exclusively through formal **Pull Requests (PRs)** accompanied by peer review. Dumping entire codebases in a single, monolithic end-of-semester commit is strictly prohibited and will result in zero marks for individual Git contribution; maintaining an unbroken, transparent Git commit and PR audit trail is essential for passing the code ownership defense during the live viva-voce examination.

Repository README.md Specification Template

The root **`README.md`** serves as the primary technical landing page and must strictly conform to the following structural format:

````
# Project 09: Malayalam Multi-Domain Text Summarization Compression & Extractive Scorer
[![Course](https://img.shields.io/badge/Course-Programming%20in%20Python-blue.svg)](https://duk.ac.in)
[![Python](https://img.shields.io/badge/Python-3.11%2B-brightgreen.svg)](https://python.org)
[![License](https://img.shields.io/badge/License-MIT-yellow.svg)](LICENSE)

## 1. Project Overview & Domain Context
- **Institution:** Digital University Kerala (MTech AI / MSc AI)
- **Course:** M4010104 / M4020104 – Programming in Python | **Instructor:** Dr. Sunil TT
- **Dataset:** Malayalam Texts-Summary Dataset on Kaggle (https://www.kaggle.com/datasets/nigeljoetensin

## 2. System Architecture & Pipeline Design
[Provide an ASCII or Mermaid architecture flow diagram showing raw ingestion -> validation -> analyzer -

## 3. Installation & Environment Setup
```bash
git clone https://github.com/<your-team>/malayalam-summarization-evaluator.git
cd malayalam-summarization-evaluator
python -m venv venv
source venv/bin/activate  # On Windows: venv\Scripts\activate
pip install -r requirements.txt
```

## 4. Execution Instructions
### A. Running the Standalone Application
```bash
python run_pipeline.py --input data/raw/malayalam_summaries_raw.csv --output-dir data/processed --report
```
### B. Running the Development Notebook
```bash
jupyter notebook notebooks/development_cycle.ipynb
```

## 5. Key Analytical & Statistical Findings (Q1–Q12 Summary)
[Provide a concise markdown summary table detailing raw vs. clean counts, regression coefficients, anoma

## 6. Interactive Telemetry Dashboard
Explore the live interactive Plotly visualization hosted on GitHub Pages:
🔗 [Live Project Dashboard](https://<your-team>.github.io/malayalam-summarization-evaluator/interactive_
````

```
## 7. Contributor Git Commit & Pull Request Log Table
| Contributor Name | GitHub Handle | Assigned Module / Tasks | Verified Commit Hash Range | Merged Pull
| :--- | :--- | :--- | :--- | :--- |
| [Student Full Name] | `@github_handle` | Modular Architecture (`src/ingestion.py`, `src/analyzer.py`,
```

6. Evaluation Rubric & Mandatory Academic Integrity Governance

Projects are evaluated on a 25-mark scale based on code quality, mathematical correctness, visualization depth, git collaboration discipline, and live technical defense:

Assessment Criterion	Basis of Evaluation & Deliverable Verification	Max Marks
1. Data Curation & File I/O Pipeline (Q1–Q2)	Robust line-by-line file reading with defensive **`try-except`** handling, physical/domain bounds validation, clean CSV output, and structured **`summarization_audit.log`** reporting.	**5 Marks**
2. OOP, NumPy & Pandas Analytics (Q3–Q9)	Modular **`MalayalamSummaryEvaluator`** class architecture, vectorized NumPy feature re-derivations (verified via **`np.allclose`**), regression modeling, and Pandas rolling/groupby/melt operations.	**6 Marks**
3. Multi-Library Visualizations (Q10–Q12)	Publication-quality Matplotlib confidence envelope/trend plot, Seaborn correlation heatmap and distribution violin plot, and standalone interactive Plotly HTML dashboard.	**5 Marks**
4. Software Packaging, GitHub & Documentation	Clean directory layout, comprehensive **`README.md`**, working GitHub Pages site, **`development_cycle.ipynb`** notebook, standalone **`run_pipeline.py`**, and balanced individual Git commits/PRs.	**4 Marks**
5. Live Presentation & Code Viva-Voce Defense	Technical walkthrough and individual live viva defense where each student must explain or modify lines of code in real time without external assistance.	**5 Marks**

> **STRICT ACADEMIC INTEGRITY & ZERO-TOLERANCE AI POLICY (0 MARKS PENALTY):** All submitted Python source code (**`.py`**, **`.ipynb`**) and written documentation will be screened through an automated **AI-Code & Plagiarism Detection Suite**. If AI-generated code or synthetic documentation is detected at any checkpoint or final submission, **0 Marks will be awarded immediately** for the entire project. Each student must prove authentic individual code ownership during the live viva-voce examination — *Course Instructor: Dr. Sunil TT*.

7. Milestone Schedule, Submission Checklist & Viva Defense Prompts

To ensure systematic progress across the semester, teams must adhere to three project checkpoints and prepare for on-the-spot live code modification challenges during the viva-voce examination:

Milestone Stage	Scheduled Window	Required Deliverables & Verification Milestones
Checkpoint 1 (Data Ingestion)	Week 11 Lab Session	Repository initialized with contributors and GitHub Pages active. **`src/ingestion.py`** operational with line-by-line filtering, **`clean_malayalam_summaries.csv`** exported, and **`summarization_audit.log`** generated.
Checkpoint 2 (Analytics & OOP)	Week 13 Lab Session	**`src/analyzer.py`** complete with **`MalayalamSummaryEvaluator`** class, NumPy feature checks (Q4), outlier masks (Q5), regression modeling (Q6), and exploratory notebook sections.

Milestone Stage	Scheduled Window	Required Deliverables & Verification Milestones
Final Submission & Viva	Week 15 Project Review	Full package operational: standalone **`run_pipeline.py`**, polished **`development_cycle.ipynb`**, interactive **`interactive_dashboard.html`** on GitHub Pages, 3–4 page PDF report, and live code defense.

Representative Live Viva-Voce Code Modification Challenge

During the individual code viva-voce, the instructor will ask students to execute an immediate 2-minute code modification on their working repository to demonstrate genuine authorship:

- **Challenge Prompt:** *"In src/analyzer.py, adjust the extractive sentence selection quota from top 2 to top 3 sentences. Re-run your evaluation module and report the change in token F1 overlap score."*